

ECDAT: Unique Value & Technical Differentiators

What makes our cryptographic transition platform different from existing tools

Document ID: ECDAT-USP-2026 | Audience: Security Architecture & Engineering Teams | Classification: Confidential

1. Executive summary

Most security tools built for cryptographic inventory operate as passive scanners. They search for known algorithm names, dump hundreds of findings into a spreadsheet, and leave security teams with the impossible task of triaging them manually. They treat cryptography as a static compliance checklist rather than an interconnected operational system.

ECDAT is designed around how software engineering teams actually work. It connects code analysis with database lifespans, service dependency mapping, network transport checks, and developer sprint budgets. Instead of handing you a list of 500 unprioritized problems, it identifies the most effective fixes that eliminate the most risk within a given development window.

2. Our core unique selling propositions

Six specific architectural capabilities separate ECDAT from both commodity code scanners and legacy enterprise tools:

USP 1: Intent-aware noise elimination (DSIS)

The primary reason developers ignore security scanners is alert fatigue. Traditional scanners cannot tell the difference between an SHA-256 hash used to build a 60-second Redis cache key and an RSA key encrypting customer records. Both get flagged as high-priority findings.

ECDAT traces data flow to its terminal destination. Cryptographic operations that terminate in local memory, cache lookups, or internal logging are classified as operational utilities. Their risk score is set to zero, automatically suppressing alerts. Only operations that protect persistent databases, credentials, or network egress trigger migration warnings. In testbeds, this cuts out 90% to 95% of false alerts, leaving developers with only actionable security issues.

USP 2: Dependency contagion & superspreader detection

Traditional tools look at files in isolation. If a central authentication service issues tokens using RSA-2048, and twenty microservices verify those tokens, a standard scanner flags all twenty-one services as twenty-one separate, equal vulnerabilities.

ECDAT builds a directed dependency graph across your repositories and services. It identifies the provider-consumer relationship and calculates a blast radius for every cryptographic component. Components that supply tokens, certificates, or session keys to multiple downstream applications are tagged as 'superspreaders'. Migrating a single superspreader automatically protects all downstream consumer services. This allows security leads to target the root cause rather than wasting time fixing symptoms.

USP 3: Real lifespan modeling from database schemas

Michele Mosca's migration theorem requires knowing the data lifespan (X). Existing platforms plug in a flat, arbitrary ten-year guess for every asset in an organization. This produces misleading risk timelines.

ECDAT inspects actual database schemas, table retention rules, and column constraints to estimate how long data will stay in storage. An authentication cookie is recognized as short-lived, while an encrypted tax or medical column receives a multi-year retention profile. It then runs a 5,000-iteration stochastic simulation to calculate an empirical breach probability curve, showing security leaders exactly when their data retention crosses into the quantum risk window.

USP 4: Transport MTU & memory buffer safety checks

This is a major blindspot across existing commercial tools. Post-quantum cryptographic keys and signatures are massive compared to classical cryptography. An ECDSA signature is 64 bytes; an ML-DSA-65 signature is 3,309 bytes. An RSA public key is 256 bytes; an ML-KEM-768 public key is 1,184 bytes.

Blindly telling developers to upgrade their algorithms causes two severe outages in production:

- Network packet fragmentation: A 5 KB post-quantum handshake flight exceeds standard 1,500-byte Ethernet MTUs. Middleboxes, firewalls, and cloud load balancers frequently drop fragmented handshake packets, silently breaking API connections. ECDAT tests network socket segment limits (MSS) and warns teams to configure certificate compression before deploying.
- Memory buffer truncation: Legacy applications and embedded systems often allocate fixed 64-byte or 256-byte buffers to hold signatures. Injecting an ML-DSA signature causes buffer overflows or silent truncation. ECDAT scans syntax trees for fixed-size allocations and flags memory hazards before code hits production.

USP 5: Sprint-budgeted remediation planning (Pareto knapsack)

No engineering organization has unlimited time to fix every cryptographic flaw at once. A scan report with 500 'Critical' items creates analysis paralysis.

ECDAT translates abstract risk into concrete developer-weeks. Security managers input their team's available sprint budget (for example, four developer-weeks). ECDAT calculates the migration effort for each asset and ranks the backlog using an efficiency-ratio algorithm (risk reduction divided by development effort). It outputs an optimal remediation queue and an interactive Pareto frontier curve, showing management the exact subset of work that yields the maximum risk reduction for that specific sprint.

USP 6: Tamper-evident attestation & audit ledger

Most scanners export plain JSON or CSV files. Anyone with text-editor access can delete vulnerable rows or alter algorithm names to fake audit compliance.

ECDAT wraps all inventory results in an in-toto compliant Dead Simple Signing Envelope (DSSE) signed with an Ed25519 key. Concurrently, it computes a 32-byte SHA-256 Merkle root across all findings. External compliance auditors can mathematically verify whether a specific component was part of an approved scan, or prove that an unapproved algorithm is absent, without requiring access to the underlying proprietary source code.

3. How ECDAT compares against the market

The table below highlights how ECDAT's architecture addresses the gaps found in current market alternatives:

Capability	Commodity SAST	Enterprise Scanners	ECDAT Approach
Noise Filtering	None (flags all crypto)	Basic rule suppression	Intent classification (DSIS) suppresses 90%+ noise
Service Blast Radius	Isolated files only	Basic asset catalog	Dependency contagion graph (R0) pinpoints superspreaders
Lifespan Estimation	None	Static 10-year guess	Derived from database schemas + 5,000-run Monte Carlo
PQC Size & MTU Checks	None	None	Checks socket MSS & code buffers against NIST PQC byte sizes
Fix Prioritization	Flat severity list	Ticketing sync only	Pareto knapsack planner fits fixes into sprint budgets
Audit Integrity	Plain text/JSON	Proprietary database	Cryptographically signed DSSE envelopes + Merkle proofs

4. Measured accuracy & engineering integrity

We hold our code to strict empirical standards. When tested against standard academic security benchmarks, ECDAT delivered:

- CryptoAPI-Bench (182 Java test cases covering broken ciphers, static IVs, and hardcoded keys): 100% recall with zero false negatives.
- CamBench (459 cases with complex control flow and dead branches): 93.6% recall and 94.0% precision. Our dead-branch pruning reduced false alarms by 85.3% compared to baseline scanners.
- Apache Production Code (2,005 Java source files from Camel, CXF, and Santuario): 94.2% verified detection accuracy.

Engineering transparency on current implementation

We are clear about what runs in production today versus what is in active development on our roadmap:

1. Code analysis: Python is parsed via native syntax trees (ast.parse). Java uses a 53-pattern regular expression engine with backward variable tracing and dead-code blanking. Other polyglot languages (Go, Rust, JS) are mapped via lexer token streams.
2. Safe remediation: Our remediation engine operates via exact, verified string token substitutions backed by file checksums and rollback journals. It avoids compiler AST mutations that could corrupt code formatting or alter unrelated lines.
3. Runtime sensing: Live checks currently run via active network socket inspection (ecdat probe). An experimental in-kernel eBPF tracing script exists in the repository, but is maintained as a standalone tool because live kernel probes require root access.
4. Signing infrastructure: Audit envelopes are signed using Ed25519. Full post-quantum signing (ML-DSA-65) is scheduled for our next release.

5. Suggested technical pilot

To evaluate how ECDAT performs in your environment, we suggest a simple two-phase pilot on non-sensitive infrastructure:

Pilot Execution: Phase 1: Run 'ecdat scan' against a sample internal repository to review the generated inventory and verify that cache/utility hashes are automatically silenced.

Phase 2: Run 'ecdat probe' against an internal API gateway to inspect cipher suites and check whether packet segment sizes (MSS) can support post-quantum key exchanges.

The CLI installs locally in seconds, runs completely offline, and sends zero telemetry outside your corporate perimeter.